Программ хангамжийн инженерчлэлийн тусгай сэдвүүд
(Бие даалт)

Д. Батмөнх

2026 оны 3-р сарын 14

Бие даалт №1: Дэлгүүрийн кассын систем

Бодлогын үндсэн тавилт

Нэгэн супермаркет нь Улаанбаатар хотод 25 салбартай бөгөөд 1 салбартаа дунджаар 5 ширхэг кассын машин буюу компьютертой. Кассын машин бүр нь үйлчлүүлэгчийн худалдан авсан барааны нийт үнийн дүнг тооцоолохдоо барааны кодод (barcode) харгалзах үнийн мэдээллийг тухайн салбар дэлгүүрийн сервер компьютероос авч баримтыг тасалбар болгон хэвлэнэ.

Улмаар тэрхүү мэдээллээ тодорхой хугацааны мөчлөгтэйгөөр (1–5 минут буюу тасалбар хэвлэгдмэгц) тухайн салбар дэлгүүрийн серверт илгээнэ. Уг сервер нь төв сервер уруу мэдээллээ оройн 10 цагт илгээнэ.

Кассын машин бүр тухайн салбар дэлгүүрийнхээ серверийн өгөгдлийн санд үйлчлүүлэгчээс цуглуулсан мэдээллээ нэгэн зэрэг илгээнэ. Мөн салбар дэлгүүр бүрд байрлах сервер бүр төв серверийн өгөгдлийн санд нэгэн зэрэг бичилт хийнэ.

Хэрэв тухайн салбар дэлгүүрийн барааны үнийн мэдээллийг өөрчлөх шаардлагатай гэж үзвэл ердөө төв серверийн өгөгдлийн санд өөрчлөлт хийхэд хангалттай.

Баярын өдрүүдээр үйлчлүүлэгчдийн тоо эрс өсдөг бөгөөд энэ үед кассаас ирэх хүсэлтийн тоо дэд серверийн ачааллыг эрс өсгөж, системийг удаашруулдаг аж.

Жилийн эцсийн тооллогоор нийт борлуулсан барааны тоо ширхэг болон орлого зарлагад зөрөө гардаг байна. Шалтгааныг тодорхойлох зорилгоор борлуулалтын алба болон мэдээлэл технологийн албаны хамтарсан багийн нарийвчилсан шалгалтаар нийт алдагдлын 63.5% хувь нь программын бүртгэлтэй холбоотой, 31.4% нь өөртөө үйлчлэх дэлгүүрүүдийн үйлчлүүлэгчдийн хяналт аюулгүй байдлын доголдлоос үүдэлтэй, үлдэх хувь нь агуулахын алдаатай тооллоготой холбоотой байсан ажээ.

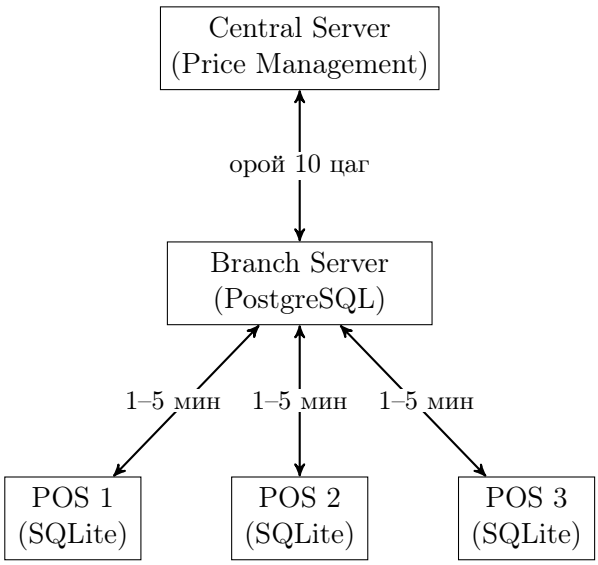
Мэдээлэл технологийн албаныхан программтай холбоотой дээрх алдааг

дараах байдлаар дүгнэж байна:

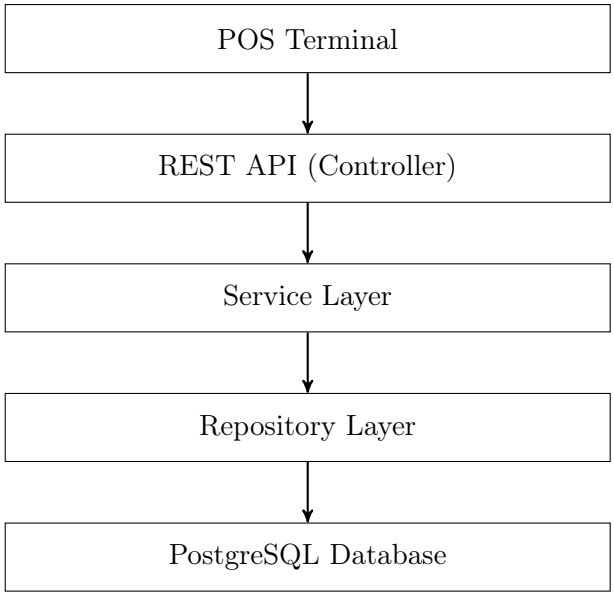
1. Өрсүүлэлттэй холбоотой асуудлыг шийдэхгүй, хэдийгээр кодод өрсүүлэлт ашиглаагүй ч өгөгдлийн санд энэ асуудал гарч байна
2. Оргил ачааллын үеийн системийн гацалтыг арилгах зорилгоор өрсүүлэлт (thread) ашиглах
3. Программ хангамжийн архитектурыг сайжруулж, программ хангамжийн загварчлал (design pattern) ашиглах
4. Кодын чанарыг сайжруулах
5. Олон хөгжүүлэгчийн гар дамжсан учир код эмх замбараагүй, алдаа ихтэй
6. Шинэ боломж нэмэхэд төвөгтэй, олон газар засвар хийх шаардлагатай болдог
7. Хөгжүүлэлтийн баг туршлага багатай хөгжүүлэгчдэд кассын программыг сайжруулах даалгавар өгөхөөс эмээдэг, учир нь бодит системийг гэмтээх, гацаах аюултай гэж үздэг.

Жич: Угтаа тэдний хэн нь ч системийн бичил шалгалт хангалтгүй учраас дээрх байдалд хүрч байгааг ухаарахгүй байгаа.

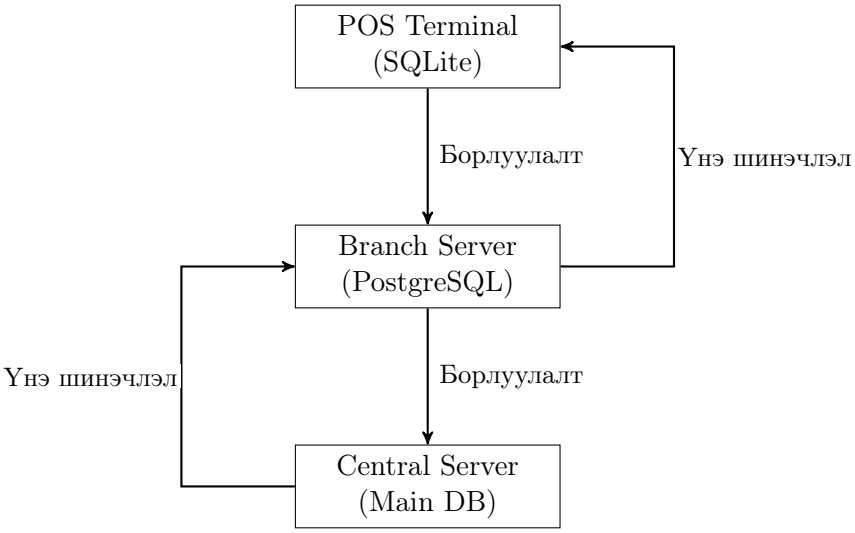
Системийн архитектур



Серверийн программын архитектур



Мэдээлэл шинэчлэгдэх (sync) урсгал



Үйлдлийн дэлгэрэнгүй

1. Бараа уншуулах. Кассчин барааны кодыг уншуулна. POS terminal (кассын компьютер) нь салбарын серверийн `GET /api/products/{barcode}`

endpoint уруу хүсэлт илгээж, барааны нэр, үнийг авна. Хэрэв сүлжээний алдаа гарвал POS өөрийн SQLite дээрх хуучин үнийг ашиглана. Хэрэв барааны кодоор хайхад тухайн бараа олдохгүй бол нэр болон төрлөөр хайж олно.

2. Борлуулалтын бүртгэл. Бүх барааг уншуулж, төлбөр төлөгдмөгц тасалбар хэвлэгдэнэ. POS terminal нь борлуулалтын мэдээллийг (барааны жагсаалт, тоо ширхэг, үнэ, нийт дүн, төлбөрийн төрөл) өөрийн SQLite өгөгдлийн санд хадгална.

3. POS болон салбарын серверийн мэдээлэл дамжуулалт POS terminal нь хадгалсан борлуулалтын мэдээллээ тасалбар хэвлэгдмэгц `POST /api/sales` endpoint уруу илгээнэ. Салбарын сервер хүлээн авч PostgreSQL өгөгдлийн санд хадгална. Амжилттай илгээсэн бол POS дээрх `synced` төлөв шинэчлэгдэнэ. Амжилтгүй болсон тохиолдолд дараагийн тасалбар хэвлэгдэхэд дахин илгээнэ.

4. Салбарын сервер болон төв серверийн мэдээлэл дамжуулалт Орой бүр 10 цагт салбарын серверүүд тухайн өдрийн борлуулалтын мэдээллээ төв сервер уруу `POST /api/sync` endpoint ашиглан илгээнэ. Ингэхдээ зөвхөн sync хийгдээгүй (`synced = false`) бичлэгүүдийг илгээнэ.

5. Үнийн мэдээлэл шинэчлэх. Төв сервер дээр барааны үнэ өөрчлөгдөхөд салбар серверүүдэд `GET /api/products/updates` endpoint ашиглан шинэ үнийн мэдээлэл sync хийгдэнэ. Салбар сервер шинэ үнийг өөрийн PostgreSQL өгөгдлийн санд хадгалж, POS машинууд дараагийн barcode уншуулахдаа шинэ үнийг авна.

6. Алдааны нөхцөл. Сүлжээ тасарвал POS өөрийн SQLite дээрх үнийн мэдээллээ ашиглана. Sync амжилтгүй болсон мэдээллүүдийг дахин илгээх (retry) тохиргоог хийгдсэн байна. Давхар илгээхээс сэргийлэхийн тулд борлуулалт бүрд дахин давтагдашгүй ID (UUID) ашиглана.

7. Мэдээллийн нөөцлөлт. POS терминал бүр өөр дээрээ сүүлийн 1 жилийн хугацааны өгөгдлийг хадгалах бөгөөд шинэ оны 01/01-д зөвхөн өмнөх 12 сарын мэдээллээ үлдээгээд бусдыг нь устгана. Харин салбарын серверүүдийн хувьд өгөгдлийн санг огноогоор бүлэглэн (partition үүсгэн) хадгална.

Жич: бие даалтын ажлын хүрээнд зарим нарийн тохиргоо болох зангилаа (node) нөөц (backup) серверүүд, өөр хоорондоо автоматаар мэдээлэл дамжуулах зэргийг тооцохгүй.

Гүйцэтгэх ажлын даалгавар

Салбарын серверийн backend системийг хөгжүүлж, RESTful API бичнэ:

1. POS машинаас илгээх борлуулалтын мэдээллийг хүлээн авах

2. Барааны кодоор барааны үнийг хайх
3. Борлуулалтын мэдээллийг өгөгдлийн санд хадгалах
4. POS серверүүд өгөгдлөө салбарын серверт өрсүүлэх аргачлалаар нэгэн зэрэг дамжуулна (зөвхөн шинэ өгөгдөл, retry бүхий)

REST API бэлтгэхдээ дараах үндсэн технологиудыг ашиглана:

- Ашиглах хэл: Java
- Бүтээц (framework): Spring Boot
- Өгөгдлийн сан: Контейнер (Docker) ашиглан PostgreSQL ажиллуулна
- REST API нь OpenAPI specification ашиглан баримтжуулсан байх
- Swagger UI ашиглан API-г турших боломжтой байх
- Request validation хэрэгжүүлсэн байх

Өгөгдлийн сангийн бүтэц

Системийн өгөгдлийн сан нь 10 хүснэгтээс бүрдэх бөгөөд доорх дүрслэлд баганын нэр, төрөл, хязгаарлалт, тайлбарыг тусгав.

Зарим товчилсон үгс:

PK — primary key,
FK — foreign key,
NN — not null,
UQ — unique,
DF — default утга.

category

Барааны ангилал

Багана	Төрөл	Хязгаарлалт	Тайлбар
`id`	`SERIAL`	`PK`	ID тоолуур
`name`	`VARCHAR(100)`	`NN`, `UQ`	Ангиллын нэр

product

Барааны үндсэн мэдээлэл

Багана	Төрөл	Хязгаарлалт	Тайлбар
id	SERIAL	PK	ID тоолуур
name	VARCHAR(255)	NN	Барааны нэр
category_id	INTEGER	FK → category.id, NN	Ангиллын ID
created_at	TIMESTAMP	NN, DF NOW()	Бүртгэсэн огноо
updated_at	TIMESTAMP	NN, DF NOW()	Сүүлд өөрчилсөн огноо
active	BOOLEAN	NN, DF TRUE	Идэвхтэй эсэх

barcode

Нэг бараанд олон barcode харгалзаж болно

Багана	Төрөл	Хязгаарлалт	Тайлбар
id	SERIAL	PK	ID тоолуур
product_id	INTEGER	FK → product.id, NN	Барааны ID
code	VARCHAR(50)	NN, UQ	Barcode утга (EAN-13 г.м.)

product_price

Барааны үнийн түүх. Хамгийн сүүлийн бичлэг нь одоогийн үнэ

Багана	Төрөл	Хязгаарлалт	Тайлбар
id	SERIAL	PK	ID тоолуур
product_id	INTEGER	FK → product.id, NN	Барааны ID
price	NUMERIC(12,2)	NN, CHECK ≥ 0	Үнэ (MNT)
updated_at	TIMESTAMP	NN, DF NOW()	Үнэ тогтоосон огноо

UNIQUE(product_id, updated_at) — нэг бараанд нэг мөчид давхар үнэ оруулахаас сэргийлнэ

branch

Салбар дэлгүүр

Багана	Төрөл	Хязгаарлалт	Тайлбар
id	SERIAL	PK	ID тоолуур
name	VARCHAR(150)	NN, UQ	Салбарын нэр
address	VARCHAR(300)	NN	Хаяг

pos_terminal

Кассын машин (POS)

Багана	Төрөл	Хязгаарлалт	Тайлбар
id	SERIAL	PK	ID тоолуур
branch_id	INTEGER	FK → branch.id, NN	Салбарын ID
name	VARCHAR(50)	NN	POS нэр

payment_type

Төлбөрийн төрөл (бэлэн мөнгө, карт, QR г.м.)

Багана	Төрөл	Хязгаарлалт	Тайлбар
id	SERIAL	PK	ID тоолуур
name	VARCHAR(50)	NN, UQ	Төлбөрийн нэр

discount_rule

Хөнгөлөлтийн дүрэм

Багана	Төрөл	Хязгаарлалт	Тайлбар
id	SERIAL	PK	ID тоолуур
name	VARCHAR(100)	NN	Дүрмийн нэр
type	VARCHAR(20)	NN	PERCENT / FIXED
value	NUMERIC(12,2)	NN, CHECK ≥ 0	Хөнгөлөлтийн утга
active	BOOLEAN	NN, DF TRUE	Идэвхтэй эсэх

sales

Борлуулалтын гүйлгээ

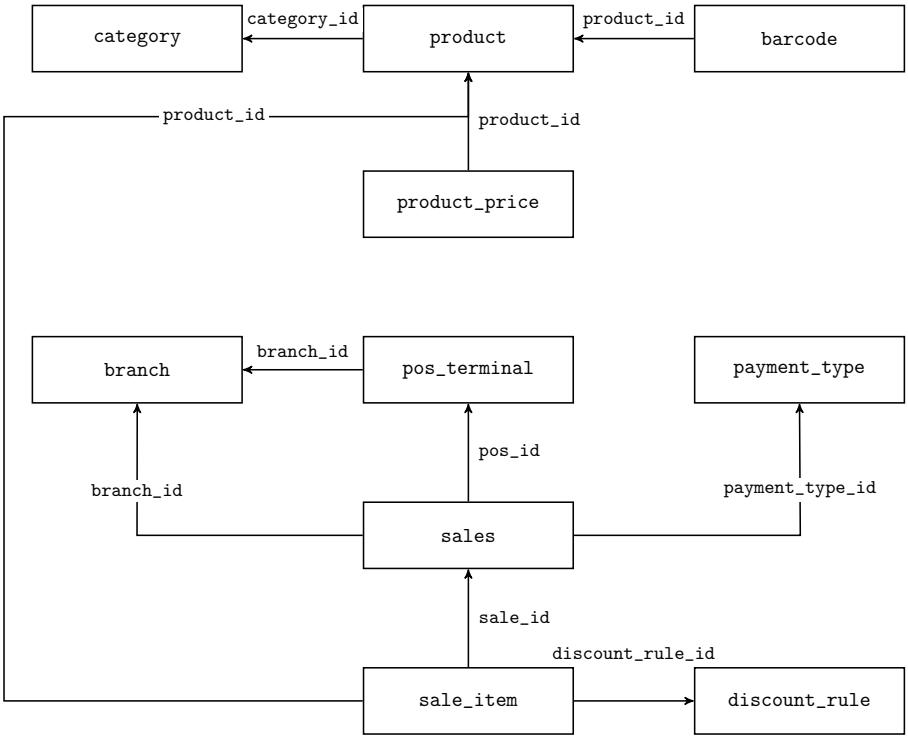
Багана	Төрөл	Хязгаарлалт	Тайлбар
id	UUID	PK	Давтагдашгүй ID
branch_id	INTEGER	FK → branch.id, NN	Салбарын ID
pos_id	INTEGER	FK → pos_terminal.id, NN	POS машины ID
payment_type_id	INTEGER	FK → payment_type.id, NN	Төлбөрийн төрөл
total_amount	NUMERIC(14,2)	NN, CHECK ≥ 0	Нийт дүн (MNT)
created_at	TIMESTAMP	NN, DF NOW()	Борлуулалтын огноо
synced	BOOLEAN	NN, DF FALSE	Sync хийгдсэн эсэх

sale_item

Борлуулалтын мөр (нэг гүйлгээний бараа тус бүр)

Багана	Төрөл	Хязгаарлалт	Тайлбар
`id`	`SERIAL`	`PK`	ID тоолуур
`sale_id`	`UUID`	`FK` → `sales.id`, `NN`	Борлуулалтын ID
`product_id`	`INTEGER`	`FK` → `product.id`, `NN`	Барааны ID
`discount_rule_id`	`INTEGER`	`FK` → `discount_rule.id`	Хөнгөлөлт (NULL боломжтой)
`quantity`	`INTEGER`	`NN`, `CHECK` > 0	Тоо ширхэг
`price`	`NUMERIC(12,2)`	`NN`, `CHECK` ≥ 0	Зарагдсан үнэ
`discount_amount`	`NUMERIC(12,2)`	`NN`, `DF 0`	Хөнгөлөлтийн дүн

Entity-Relationship (ER) диаграмм



REST API

API contract. REST API нь JSON формат ашиглаж, endpoint бүр дараах дүрмийг баримтална:

- Request болон response нь `application/json` төрөлтэй байна
- Амжилттай үйлдэлд `200 OK` эсвэл `201 Created` буцаана
- Алдаатай хүсэлтэд `400 Bad Request` буцаана
- Нөөц олдохгүй бол `404 Not Found`
- Давхар хүсэлт илэрвэл `409 Conflict`

Алдааны хариулт дараах бүтэцтэй байна:

```
{
  "error": "message",
  "timestamp": "2026-03-12T10:00:00Z"
}
```

Method	Endpoint	Status	JSON Structure
GET	`/api/products/{barcode}`	`200 OK`	`{ id, name, barcode, price }`
GET	`/api/products/search`	`200 OK`	`{ query, category, page, size }`
POST	`/api/sales`	`201 Created`	`{ branch_id, pos_id, payment_type, items[], total_amount }`
GET	`/api/sales/{id}`	`200 OK`	`{ id, branch_id, pos_id, items[], total_amount, created_at }`
POST	`/api/sync`	`200 OK`	`{ branch_id, sales[] }`

POST /api/sales endpoint contract. Шинэ борлуулалтын мэдээллийг серверт хадгална. Request JSON:

```
{
  "id": "UUID",
  "branch_id": 1,
  "pos_id": 2,
  "payment_type_id": 1,
  "items": [
    {
      "product_id": 10,
      "quantity": 2,
      "price": 4500
    }
  ],
  "total_amount": 9000
}
```

Response:

```
201 Created
{
  "id": "UUID",
  "status": "stored"
}
```

Өрсүүлэлт

Салбар бүрийн барааны үлдэгдэл:

Багана	Төрөл	Хязгаарлалт	Тайлбар
`id`	`SERIAL`	`PK`	ID
`branch_id`	`INTEGER`	`FK` → `branch.id`	Салбар
`product_id`	`INTEGER`	`FK` → `product.id`	Бараа
`quantity`	`INTEGER`	`NN`, `CHECK >= 0`	Үлдэгдэл

Өрсүүлэлтийг программын болон өгөгдлийн сангийн төвшинд зохион байгуулна:

1. Олон POS зэрэг борлуулалт илгээх. Нэг салбарт 5 POS нэгэн зэрэг `POST /api/sales` endpoint уруу борлуулалтын мэдээлэл илгээнэ. Хэрэв данс (inventory) хөтөлдөг бол хоёр POS ижил барааг зэрэг зарахад данс буруу тоологдож болно. Үүнийг database transaction, `SELECT ... FOR UPDATE` ашиглан шийднэ.

2. Олон салбар зэрэг sync хийх. 25 салбар сервер төв сервер уруу зэрэг sync хийхэд ачаалал нэмэгдэнэ. Thread pool ашиглан зохицуулна.

3. Үнэ шинэчлэгдэж байхад борлуулалт хийх. POS хуучин үнээр бараа зарж болно. Аль үнээр зарагдсаныг `sale_item.price` талбарт хадгална.

Design Pattern

Дараах загваруудыг ашиглана:

- Repository pattern — өгөгдлийн сангийн давхаргыг бизнес логикоос тусгаарлах
- Strategy pattern — төлбөрийн өөр өөр арга (бэлэн, карт, QR) болон шинэ арга нэмэхэд код өөрчлөхгүй (OCP)

- Observer pattern — үнэ өөрчлөгдөхөд POS-уудад мэдэгдэх, sync амжилттай болоход log бичих

Clean Code ба SOLID

SOLID — ялангуяа OCP: шинэ төлбөрийн төрөл, хямдралын нөхцөл, баримтын загвар нэмэхэд код өөрчлөхгүй. Чанартай кодын зарчмыг баримтална:

- Нэр сонгох
- Функц бичих
- Алдааг боловсруулах
- Өрсүүлэлт

Бичил шалгалт

JUnit ашиглан бичил шалгалт бичнэ. Үзүүлэлтийг шалгахдаа JaCoCo ашиглах бөгөөд дараах шаардлагыг хангасан байна:

1. line coverage $\geq$ 70% — кодын мөрийн хамралт
2. method coverage $\geq$ 70% — функцийн хамралт

Ачааллын шалгалт[1]

Системийн гүйцэтгэлийг шалгахын тулд ачааллын шалгалт (load test) хийж үр дүнг тайлагнана.

Энэ шалгалтын зорилго нь олон POS терминалаас нэгэн зэрэг ирэх хүсэлтүүд сервер дээр ямар нөлөө үзүүлж байгааг шалгах явдал юм.

Туршилтын нөхцөл:

- 1 салбарт 5 POS терминал бий
- 25 салбар зэрэг ажиллана
- Нийт 125 POS терминал ажиллана
- POS бүр 1 секунд тутамд 1 хүсэлт илгээнэ

[1] load test

Ингэснээр систем секундэд ойролцоогоор 125 хүсэлт хүлээн авна.

Турших endpoint:

```
/api/sales
/api/products/{barcode}
```

Ачааллын шалгалт хийхдээ дараах хэрэгслүүдийн аль нэгийг ашиглаж болно: Apache JMeter, k6 эсвэл Locust.

Туршилтын тайланд дараах үзүүлэлтүүдийг багтаана:

- 1 секундэд ирэх хүсэлтийн тоо[2]
- хариу ирэх дундаж хугацаа
- хариу ирэх хамгийн удаан хугацаа
- алдаатай хүсэлтийн хувь

Ачааллын шалгалтын үр дүнг хүснэгт эсвэл зураглал хэлбэрээр техникийн баримт бичигт оруулна. Ингэхдээ туршилт явуулсан компьютерын үзүүлэлтийг дараах командаар terminal ашиглан авна:

```
# CPU
lscpu | grep -E "Model name|MHz|cache"
cat /proc/cpuinfo | grep "MHz" | head -1

# RAM
free -h
sudo dmidecode --type 17 | grep -E "Size|Speed|Type"

# Disk
lsblk -d -o NAME,SIZE,MODEL
sudo hdparm -Tt /dev/sda

# Software
java -version
docker exec pos-postgres psql -U posuser -c "SELECT version();"
```

Жишээлбэл, тайланд бичих мэдээлэл нь дараах байдлаар харагдана:

```
Model name: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
CPU MHz: 2592.000
```

[2] requests per second

```
L1d cache: 192 KiB
L1i cache: 192 KiB
L2 cache: 1.5 MiB
L3 cache: 12 MiB
RAM: 16 GB DDR4 3200 MT/s
Disk: Samsung SSD 970 EVO Plus (920 MB/s read)
Java: OpenJDK 21
Database: PostgreSQL 16 (Docker)
```

CPU 2.59 GHz бол секундэд ойролцоогоор 2.59×10^9 clock cycle гүйцэтгэнэ. Нэг HTTP хүсэлт ойролцоогоор 100,000 CPU instruction шаарддаг гэж үзвэл CPU нь онолын хувьд секундэд ойролцоогоор 25,900 хүсэлт боловсруулах боломжтой. Харин DDR4-3200 RAM нь 3.2×10^9 transfer/sec хурдтай бөгөөд memory bus нь 64 bit өргөн тул нийт bandwidth нь ойролцоогоор 204 Gbit/sec буюу 25.6 GB/s байна.

CPU нь өгөгдлийг аль санах ойгоос авч байгаагаас хамааран хандах хугацаа (access latency) ихээхэн ялгаатай байдаг бөгөөд CPU register-аас унших нь хамгийн хурдан, харин disk-ээс унших нь хамгийн удаан байдаг.

Санах ой	Access latency
CPU register	∼1 cycle
L1 cache	∼3–4 cycle
L2 cache	∼10 cycle
L3 cache	∼30–50 cycle
RAM	∼100–150 cycle
Disk	хэдэн сая cycle

Гэхдээ бодит системийн гүйцэтгэл нь зөвхөн CPU-оос бус өгөгдлийн сан, сүлжээ болон дискний I/O-оос ихээхэн хамаардаг.

Найдваржилт[3]

Сүлжээний алдаа, timeout эсвэл retry зэргээс POS машин нэг борлуулалтын мэдээллийг давхар илгээх боломжтой. Хэрэв сервер ижил борлуулалтыг хоёр удаа хүлээн авбал өгөгдлийн санд давхар бичигдэж, нийт орлогын тооцоо буруу гарах эрстэлтэй.

Иймээс `POST /api/sales` endpoint нь алдаагүй найдвартай байх ёстой.

Борлуулалт бүр POS талд үүсгэсэн давтагдашгүй UUID ашиглана. Энэхүү `id` утга нь `sales` хүснэгтийн primary key болно.

[3] idempotency

Жишээ JSON:

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "branch_id": 1,
  "pos_id": 3,
  "payment_type_id": 2,
  "items": [
    {
      "product_id": 15,
      "quantity": 2,
      "price": 4500
    }
  ],
  "total_amount": 9000
}
```

Хэрэв сервер ижил `id`-тай хүсэлтийг дахин хүлээн авбал шинэ бичлэг үүсгэхгүй, өмнөх гүйлгээг буцаана.

Ингэснээр:

- Retry ажиллах боломжтой
- Давхар бичилтээс сэргийлнэ
- Хуваарилалтат (distributed) системийн найдвартай ажиллагаа нэмэгдэнэ

Docker

Өгөгдлийн санг бэлэн тохируулсан контейнероор ашиглах нь компьютертоо суулгаж, тохиргоо хийх ажлаас хөнгөвчлөх давуу талтай. Гэхдээ бодит сервер дээр (bare metal) суулгаж ашиглахаас гүйцэтгэл хангалтгүй байдаг гэдгийг санах хэрэгтэй. Дараах зааврын дагуу суулгана:

1. Docker суулгах

Linux (Debian) систем дээр Docker болон Docker Compose-ийг дараах командаар суулгана:

```
sudo apt update
sudo apt install docker.io docker-compose
```

Docker зөв суусан эсэхийг шалгах:

```
docker --version
docker compose version
```

2. docker-compose.yml файл үүсгэх

Төслийн root хавтсанд дараах `docker-compose.yml` файлыг үүсгэнэ:

```
cd prj1
vim docker-compose.yml
```

гэж ороод дараах тохиргооны файлыг хуулна:

```
version: "3.9"
services:
  postgres:
    image: postgres:16
    container_name: pos-postgres
    restart: always
    environment:
      POSTGRES_USER: posuser
      POSTGRES_PASSWORD: pospass
      POSTGRES_DB: posdb
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./schema.sql:/docker-entrypoint-initdb.d/schema.sql
volumes:
  postgres_data:
```

3. PostgreSQL эхлүүлэх

Дараах командаар PostgreSQL контейнерыг ажиллуулна:

```
docker compose up -d
```

Docker дараах үйлдлийг автоматаар гүйцэтгэнэ:

- PostgreSQL image татах
- контейнер үүсгэх
- өгөгдлийн санг эхлүүлэх
- `schema.sql` файлыг автоматаар ажиллуулах

4. PostgreSQL-д холбогдох

Контейнер доторх PostgreSQL-д дараах командаар холбогдоно:

```
docker exec -it pos-postgres psql -U posuser -d posdb
```

5. Контейнерыг зогсоох

Өгөгдлийн сангийн программыг зогсоох:

```
docker compose down
```

Хүлээлгэж өгөх

1. Source code
2. Docker тохиргооны файл
3. Бичил шалгалт
4. Техникийн баримт бичиг «`Branch Server Backend System - Software Design Document.pdf`» байх бөгөөд бэлтгэхдээ LibreOffice программын `template.ott` загвар файлыг ашиглана.

Төслийн хавтасны зохион байгуулалт

`https://gitlab.com/bataak/f.csa303-2526` кодын санг өөрийн нэр дээр (account) хувилбарлаад (fork), өөрийн хувилбараа заавал private болгож[4], дараах зохион байгуулалтын дагуу ажиллана:

```
prj1/
|-- docker-compose.yml
|-- schema.sql
|-- README.md
`-- students/
    |-- B2312345/
    |   |-- src/
    |   |   |-- main/
    |   |   |   `-- java/...
    |   |   |
    |   |   |-- test/
    |   |   |    `-- java/...
```

[4] `Settings > General > Visibility > Project visibility > Private`

```
|   |   |
|   |-- docs/
|   |   |-- Branch Server Backend System -- Software Design Document.odt
|   |   `-- Branch Server Backend System -- Software Design Document.pdf
|   |
|   |-- pom.xml
|   `-- README.md
|
|-- B2315678/
`-- B2319999/
```

Бие даалтаас сурах зүйл

1. Distributed system architecture
2. REST API хөгжүүлэлт
3. Concurrency асуудал шийдвэрлэх
4. Өгөгдлийн сангийн зохион байгуулалт
5. Docker ашиглан орчин бэлтгэх
6. Unit test бичих
7. Clean Code болон SOLID зарчим хэрэгжүүлэх

Үнэлгээний шалгуур

Шалгуур	Дэлгэрэнгүй	Оноо
REST API	Endpoint зөв, HTTP method/status code, request/response бүтэц	15
Concurrency	Transaction, thread safety, race condition, sync retry	20
Design Pattern	$\geq$3 pattern зөв хэрэгжүүлсэн, тайлбарласан	15
Clean Code + SOLID	Нэрлэлт, функц, алдаа зохицуулалт, OCP	20
DB + Docker	Schema зөв, transaction, `docker-compose up` ажиллах	10
Unit Test	Coverage $\geq$ 70%, тестийн чанар	10
Баримт бичиг	Архитектур, pattern, concurrency тайлбар, README, үгийн алдаа	10

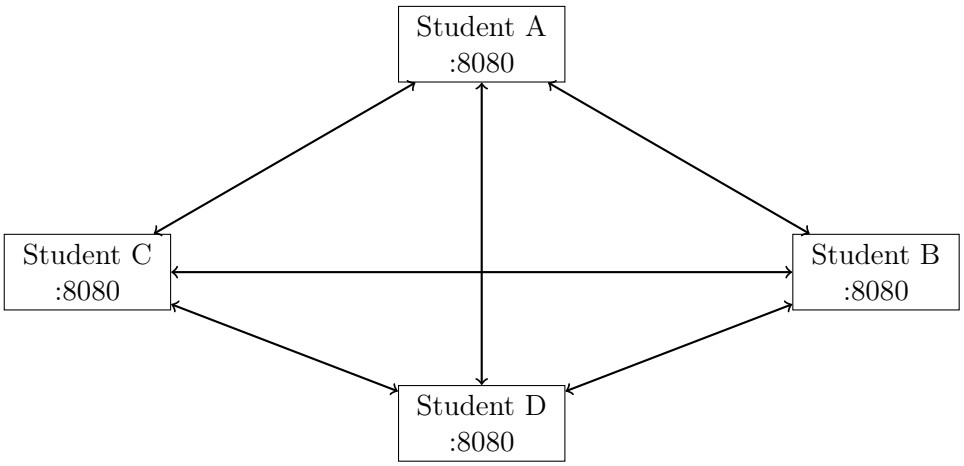
Бие даалт №2: Distributed Student Notification System

Бодлогын үндсэн тавилт

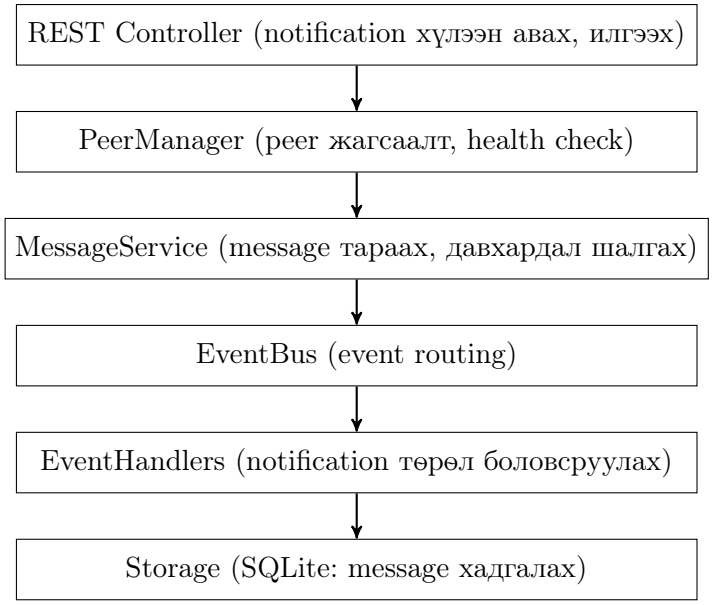
Оюутан бүр REST API ашиглан peer-to-peer distributed network үүсгэн өөр хоорондоо мэдээлэл дамжуулна. Тодруулбал:

- Оюутан бүр өөрийн notification сервер үүсгэнэ
- Серверүүд хоорондоо шууд HTTP REST API-аар холбогдоно
- Төв сервер байхгүй, message broker байхгүй, бүх node peer байдлаар ажиллана

Системийн архитектур



Node архитектур



Үйлдлийн дэлгэрэнгүй зураглал

1. Node ажиллуулах. А оюутан (peer) серверээ асаахын өмнө Б оюутны IP хаягийг config файл дотроо зааж өгөх бөгөөд сервер асахад программ А оюутны IP хаяг, портын мэдээллийг автоматаар тодорхойлж, Б оюутны peer list уруу хандаж түүний холбогдсон IP хаягийн жагсаалтыг `GET /api/peers` хүсэлтээр аваад, эргүүлээд өөрийгөө `POST /api/peers` хүсэлтээр бүртгүүлнэ.

2. Мессеж илгээх. Оюутан А "Сайн уу" гэж бичихэд сервер нь UUID, type, from, text, timestamp бүхий JSON мессеж үүсгэнэ. Энэ мессежийг `processedMessages` жагсаалтад нэмнэ. А оюутан жагсаалтаас бүх оюутнууд уруу мессежийг илгээхдээ `POST /api/notifications` хүсэлтийг нэгэн зэрэг (thread pool ашиглан) үүсгэнэ.

3. Мессеж хүлээн авах. Оюутан Б мессеж хүлээн аваад мессежийн UUID-г `processedMessages` дотроос шалгана. Аль хэдийн авсан бол үл хэрэгсэнэ. Шинэ бол `processedMessages` жагсаалтад нэмээд SQLite өгөгдлийн санд хадгалж, өөрийн жагсаалт дахь бусад оюутнууд уруу мессежийг дамжуулах.

4. Мөчлөг үүсэхээс сэргийлэх. А → В → С → А гэсэн давталтаас сэргийлж message бүрийн UUID-г `Set<String>` (thread-safe) бүтцээр хадгалж, давтагдсан мессежийг орхино.

5. Peer health check. 30 секунд тутам бүх peer уруу `GET /api/health`

илгээнэ. 3 удаа хариу өгөхгүй peer-г идэвхгүй болгоно. Дахин холбогдвол идэвхтэй болгоно.

6. Алдааны нөхцөл. Peer унтарсан бол алдааг бүртгэж, бусад peer-г шалгана. Нэг peer-ийн алдаа бусдад нөлөөлөх ёсгүй.

Мессежийн загвар

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "MESSAGE",
  "from": "192.168.1.10:8080",
  "text": "Сайн уу",
  "timestamp": 1710000000
},

{
  "id": "660e8400-e29b-41d4-a716-446655440001",
  "type": "REACTION",
  "from": "192.168.1.20:8080",
  "text": ":thumbsup:",
  "target_message_id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": 1710000060
}
```

Мэдэгдлийн (notification) төрлүүд:

- `MESSAGE` — энгийн текст, бүх peer-т харагдана
- `ANNOUNCEMENT` — чухал зарлал, онцгой тэмдэглэгээтэй хадгалагдана
- `REACTION` — тодорхой мессежид өгөх хариу

REST API

Method	Endpoint	Тайлбар
POST	`/api/notifications`	Бусад peer-ээс мессеж хүлээн авах
POST	`/api/send`	Шинэ мессеж үүсгэж бүх peer уруу тараах
POST	`/api/peers`	Шинэ peer бүртгэх
GET	`/api/peers`	Peer жагсаалт авах
DELETE	`/api/peers/{address}`	Peer хасах
GET	`/api/health`	Node холбоотой эсэхийг шалгах
GET	`/api/messages`	Хадгалсан мессежийн түүх

Өрсүүлэлт

1. Олон peer зэрэг мессеж илгээх. `processedMessages` Set уруу зэрэг бичих уралдаан (race condition) үүсэх тул `ConcurrentHashMap` эсвэл `Collections.synchronizedSet()` ашиглана.

2. Олон peer уруу зэрэг илгээх. `ExecutorService` (thread pool) ашиглан бүх peer уруу зэрэг илгээнэ. Нэг peer-ийн удаашрал бусдад нөлөөлөхгүй.

3. Peer жагсаалт зэрэг өөрчлөгдөх. Мессеж тараах явцад peer нэмэгдэж эсвэл хасагдаж болох тул жагсаалтын хуулбар үүсгэх өрсүүлэлтэд зориулагдсан `CopyOnWriteArrayList` ашиглана.

Өгөгдлийн сан (SQLite)

Node унтарвал мессеж устахаас сэргийлж SQLite ашиглана:

```
messages                    peers
--------                    -----
id (UUID)                   address (PK)
type                        status (active/inactive)
from_address                last_seen
text
timestamp
received_at
```

Node дахин эхлэхэд SQLite-аас `processedMessages` жагсаалтыг сэргээнэ.

SQLite тохиргоо. Олон thread зэрэг мессеж хүлээн аваад SQLite-д бичих үед lock алдаа гарахаас сэргийлж WAL[5] горимыг идэвхжүүлнэ:

```
PRAGMA journal_mode=WAL;
PRAGMA busy_timeout=5000;
```

WAL горим нь бичилтийг тусдаа файлд хийдэг учир нэг урсгал (thread) бичилт хийж байхад бусад урсгалуудад зэрэг уншиж чадна. `busy_timeout` нь түгжээтэй үед 5 секунд хүлээх тохиргоо юм.

Design Pattern

Программ хангамжийн дараах загварчлалыг ашиглана:

- **Strategy pattern** — мэдэгдлийн төрлүүдийг боловсруулах. Жишээлбэл, `NotificationHandler`; `MessageHandler`, `AnnouncementHandler`, `ReactionHandler`. Шинэ төрөл нэмэгдэхэд код өөрчлөхгүй (OCP)
- **Factory pattern** — төрлөөс хамаарч зөв handler үүсгэх
- **Observer pattern** — мессеж хүлээн авмагц мэдэгдэл илгээнэ

Docker

Эхлээд бичсэн программынхаа ажиллагааг турших үүднээс дараах контейнер тохиргоог ашиглан `docker-compose.yml` файлд 3–4 node тодорхойлж нэг Docker network дотор ажиллуулна:

```
services:
  node1:
    build: .
    ports: ["8081:8080"]
    environment:
      SEED_PEERS: "node2:8080,node3:8080"
    networks: [p2p-net]
  node2:
    build: .
    ports: ["8082:8080"]
    environment:
      SEED_PEERS: "node1:8080,node3:8080"
    networks: [p2p-net]
```

[5] Write-Ahead Logging

```
  node3:
    build: .
    ports: ["8083:8080"]
    environment:
      SEED_PEERS: "node1:8080,node2:8080"
    networks: [p2p-net]
networks:
  p2p-net:
    driver: bridge
```

Сүлжээний тохиргоо (бодит туршилт)

Оюутнууд өөр хоорондоо холбогдохын тулд дараах нөхцөлийг хангасан байна:

1. Бүх оюутан нэг дотоод сүлжээнд (LAN) холбогдсон байх
2. Firewall дээр 8080 портыг нээсэн байх:

   ```
   sudo ufw allow 8080/tcp
   ```

3. Өөрийн IP хаягийг мэдэх:

   ```
   ip addr show | grep "inet "
   ```

4. VPN унтраасан байх

Хүлээлгэж өгөх

1. Source code
2. Docker тохиргооны файл
3. Бичил шалгалт
4. Өмнөх бие даалтын шаардлагуудыг давхар хангасан байна
5. Техникийн баримт бичиг «`Distributed Student Notification System - Software Design Document.pdf`» байх бөгөөд бэлтгэхдээ LibreOffice программын `template.ott` загвар файлыг ашиглана.

Үнэлгээний шалгуур

Шалгуур	Дэлгэрэнгүй	Оноо
REST API	Endpoint зөв, JSON формат	15
Peer communication	Peer discovery, health check, бүртгэл/хасалт	15
Distributed messaging	Message тараах, UUID давхардал, loop хамгаалалт	15
Concurrency	Thread-safe collection, thread pool, race condition	10
Design Pattern	$\geq$3 pattern зөв хэрэгжүүлсэн, тайлбарласан	15
Clean Code + SOLID	Нэрлэх, функц, алдаа зохицуулалт, SRP, OCP, DIP	15
Unit Test	Coverage $\geq$ 70%, тестийн чанар	10
Баримт бичиг	Архитектур, pattern, concurrency тайлбар, README, үгийн алдаа	5